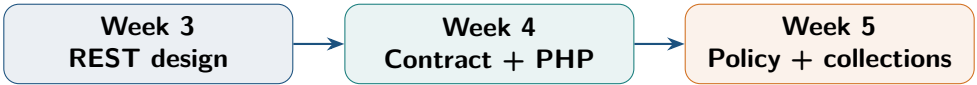
420-511-VA Web Services

Week 5
Middleware and Collection APIs

Filtering, Sorting, and Pagination

Pravish Sainath

From a Working API to a Useful Collection



A familiar question

Alex wants open Fall sections with seats available, ordered by availability, two at a time. The server should validate that request consistently.

Takeaway: Middleware organizes shared request/response behavior. Collection design organizes the returned data.

What You Will Be Able to Explain

1. Trace middleware on the request path, response path, and early-return path.
2. Design a documented filter and sort vocabulary, rather than accepting arbitrary SQL.
3. Compute a page, its metadata, and navigation links from one ordered result set.
4. Explain the trade-offs of offset and keyset/cursor pagination.
5. Connect OpenAPI, PHP validation, PDO queries, indexes, and tests.

A Familiar Interface Hides an API Contract

Find a course section

Term: Fall 2026 | **State: open** | **At least 1 seat**

Sort: most available seats first; then section ID

S102 Web Services 4 seats S103 Databases 4 seats

Page 1 of 3 **2 matching sections per page** **Next →**

```
GET /v2/sections?term=F26&state=open&minSeats=1
    &sort=-availableSeats,id&page=1&perPage=2
```

Takeaway: Every control has a precise meaning in the API contract.

The request target is shown wrapped for readability; it is one continuous URI. All data is synthetic.

Our Shared Dataset

ID	Course title	Term	State	Seats available
S101	Web Services	F26	open	1
S102	Web Services	F26	open	4
S103	Databases	F26	open	4
S104	Networks	F26	closed	5
S105	Web Services	W27	open	5
S106	Databases	F26	open	0
S107	Networks	F26	open	2
S108	Distributed Systems	F26	open	4

Takeaway: The same eight rows will make filtering, ordering, and page boundaries visible.

S101–S108 are new demonstration IDs. availableSeats = capacity minus seats_taken, as in Week 4.

Do Not Silently Truncate Week 4 Clients

Week 4: small, unpaginated v1

```
GET /v1/sections?term=F26
{"items": [/* all matching sections */]}
```

Week 5: explicit v2 collection

```
GET /v2/sections?term=F26&page=1
{"items": [...], "page": {...},
 "links": {"next": "..."}}
```

Takeaway: Adding pagination can break clients that expected the whole collection. We use a new teaching version.

This is an intentional contract change, not a claim that every new query parameter needs a new major version.

Google AIP-158: pagination

1. Middleware

One shared request pipeline; predictable behavior across routes

Why Not Repeat the Same Code Everywhere?

Without a shared layer

- Every controller starts a timer.
- Every route repeats request checks.
- Errors and response headers drift.
- A forgotten check becomes a defect.

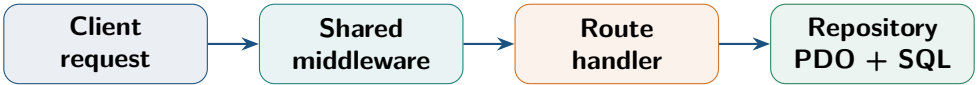
With middleware

- Common policy surrounds handlers.
- Each component has one clear job.
- Controllers focus on the operation.
- Tests can target policy separately.

Takeaway: Middleware moves repeated HTTP concerns out of individual endpoint handlers.

PHP-FIG: PSR-15

The Middleware Mental Model



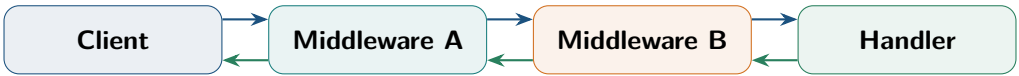
A reusable checkpoint

A component can inspect a request, pass it on, inspect the returned response, or produce an early response. It does not need to know every controller.

PHP-FIG: PSR-15

Requests Go In; Responses Come Back

request: A before → B before → handler



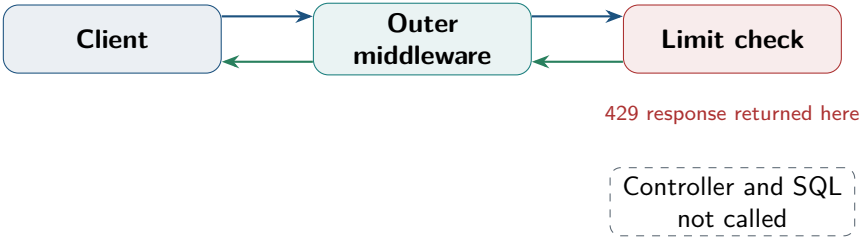
response: handler → B after → A after

```
A: before
  B: before
    handler returns a response
  B: after
A: after
```

Takeaway: Response processing unwinds the call stack; it is not a second forward pass.

PHP-FIG: PSR-15 | Slim 4: middleware order

An Early Response Skips the Inner Handler



- A rate limit, authentication decision, or cached hit can end the inner path.
- Already-entered outer middleware can still process the returned response.

PHP-FIG: PSR-15

Good Middleware Jobs and Bad Boundaries

Concern	Where it belongs	Example
Request ID / timing	Shared middleware	Attach an ID; record total duration.
Authentication	Shared or route-group policy	Establish a trusted caller identity.
Query normalization	Collection-route middleware	Parse page, filters, and sort once.
Seat allocation	Domain/repository transaction	Atomic capacity check and enrollment.
SQL joins	Repository	Build the Section representation.

Takeaway: Repeated HTTP policy belongs in middleware; domain invariants do not disappear into it.

PSR-7 and PSR-15: Roles, Not Frameworks

PSR-7: HTTP messages

- Request and response interfaces.
- Methods, URI, headers, body streams.
- withHeader / withAttribute return a changed message.

PSR-15: processing

- Handler: request in, response out.
- Middleware: request plus next handler.
- process() may delegate or return early.

Takeaway: PSR means PHP Standards Recommendation. These interfaces support interoperability; Slim is one implementation framework.

PHP-FIG: PSR-7 | PHP-FIG: PSR-15

A Middleware Component in PHP


```php
use Psr\Http\Message\ServerRequestInterface as Request;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Server\RequestHandlerInterface as Handler;
use Psr\Http\Server\MiddlewareInterface;
final class RequestIdMiddleware implements MiddlewareInterface {
    public function process(Request $request, Handler $next): Response {
        $id = bin2hex(random_bytes(8));
        $request = $request->withAttribute('requestId', $id);
        $response = $next->handle($request);
        return $response->withHeader('X-Request-ID', $id);
    }
}
```

Takeaway: Generate an ID for correlation, not authentication. Assignment captures the changed message.

Requires PSR interfaces and a PSR-7 implementation. Body streams can remain mutable.

PHP-FIG: PSR-7 | PHP-FIG: PSR-15

The Important Change from Week 4

Week 4 response helper

```
http_response_code(200);
header("Content-Type: application/json");
echo $json;
exit;
```

Middleware-compatible handler

```
$response->getBody()->write($json);
return $response
    ->withStatus(200)
    ->withHeader('Content-Type',
                 'application/json');
```

Takeaway: Return the response so outer middleware can finish. Emit it once at the application boundary.

The repository can be retained. Controllers/helpers that exit must be adapted before joining this pipeline.

PHP-FIG: PSR-15

Ordering Changes What Can Be Observed

Dependency	Required execution relationship
Error conversion	Wrap processing whose exceptions it must catch.
Route-based permissions	Resolve the route before checking its metadata.
Identity-based rate limit	Establish identity before applying that limit.
Parsed body validation	Parse the body before using its fields.
Final-status logging	Observe the converted response, not only raw exceptions.

Takeaway: Draw the actual nesting. There is no universal ordering list that fits every framework.

PHP-FIG: PSR-15 | Slim 4: error middleware

An Exception Can Bypass After-Code


```
// Outer error handler:
try { return $next->handle($request); }
catch (Throwable $e) { return aSafeProblemResponse($e); }

// Inner request-ID middleware:
$response = $next->handle($request); // throws here
return $response->withHeader('X-Request-ID', $id); // skipped!
```

Consequence

The outer error response does not automatically receive headers that an inner component would have added. Make error formatting add them explicitly, or use a carefully guarded outer finalizer.

Takeaway: Use finally for guaranteed cleanup; design error responses and observability together.

PHP-FIG: PSR-15 | Slim 4: error middleware

Slim 4: Last Added, First Entered


```php
$app->get('/v2/sections', $listSections);
$app->add(new RequestIdMiddleware());
$app->addRoutingMiddleware();
$app->addErrorMiddleware(false, true, true);

// Request: Error -> Routing -> RequestId -> handler
// Response: handler -> RequestId -> Routing -> Error
$app->run();
```

One explicit framework rule

This is Slim 4 LIFO registration. Another dispatcher may process a queue in insertion order. Always verify the chosen framework.

Bootstrap excerpt: handler, autoloading, and imports are supplied by the application. No login or CORS implementation is implied.

Slim 4: middleware order | Slim 4: error middleware

Scope: Global, Group, or One Route?

Scope	Suitable policy	Example
Application-wide	Safe exception conversion	All responses follow one error convention.
Route group	A trusted-user requirement	Enrollment routes require a caller.
One collection route	Query normalization	Only section-list routes accept page/perPage.
One operation	Special precondition	An export request has a stricter quota.

Takeaway: Keep a section filter away from unrelated enrollment creation or health-check routes.

Slim 4: middleware order

Security Policies Are Different Decisions

Different questions

- Authentication: who is the caller?
- Authorization: which rows may they access?
- CORS: may browser JavaScript read this response?
- Rate limiting: how much work is allowed?

Practical cautions

- CORS is not authentication.
- A query string is not proof of identity.
- A per-process counter is not a global quota.
- Never log tokens or confidential query values.

Takeaway: Private collection access must be enforced server-side, even when the UI hides controls.

OWASP: REST security | RFC 9110: HTTP semantics

Middleware Checkpoint

Predict before revealing the trace

```
Outer A -> Validator B -> Handler

Case 1: B calls the handler; handler returns 200.
Case 2: B returns 400 without calling the handler.
```

Reasoned answer

- Case 1: A-before, B-before, handler, B-after, A-after.
- Case 2: A-before, B rejects, A-after; handler never runs.
- An exception differs from an ordinary returned error response.

2. Filtering Collections

Choose which resources belong in the result, without changing them

Three Controls, Three Different Questions

Control	Question	School-portal example
Filtering	Which resources match?	Fall sections, open, at least one seat.
Sorting	In what order?	Available seats descending; ID ascending.
Pagination	Which bounded slice?	Page 2, two sections per page.
Field selection	Which properties?	IDs and titles only; a separate concept.

Takeaway: Filtering selects rows. Projection selects fields. Sorting orders rows. Pagination takes a slice.

Specify the Result Before Writing SQL



A semantic model

If R is the authorized set, F the predicate, O the order, and P the page slice:

$$\text{items} = \text{represent}\big(P(O(F(R)))\big).$$

Takeaway: This specifies the result; the database optimizer may choose a different physical execution plan.

The Order of These Operations Matters

Correct on our dataset

- Filter Fall, open, at least 1 seat: 5 rows.
- Sort those 5 rows.
- Take 2 rows for the selected page.

Incorrect shortcut

- Take 2 arbitrary database rows first.
- Then remove nonmatching rows.
- You may return one row and omit better matches.

Takeaway: Never paginate first and apply business filters afterward. The same warning applies to authorization.

Define a Small Query Language

Parameter	Accepted value	Meaning / omission
term	F26, W27, S27 syntax	Exact term; absent means all terms.
state	open or closed	Exact state; absent means both.
minSeats	Integer 0–500	Available seats at least this number.
sort	id, title, availableSeats	Comma-separated keys; minus means descending.
page	Integer 1–10000	Default 1.
perPage	Integer 1–100	Default 20; higher values rejected.

Takeaway: These names and limits are our contract, not mandatory REST syntax.

Query names are case-sensitive. Empty, duplicate, array-style, and unknown parameters are rejected with 400.

Equality, Ranges, and Combined Conditions


```
GET /v2/sections?term=F26&state=open&minSeats=1
```

The predicate

Keep a section if term is F26 **AND** state is open **AND** capacity minus seats taken is at least 1.

S101 **S102** **S103** **S107** **S108**

S104 is closed; S105 belongs to W27; S106 has zero seats.

Takeaway: AND versus OR must be documented. Commas do not have a universal filter meaning.

Filtering Does Not Reserve a Seat

A collection query

```
GET /v2/sections?minSeats=1
```

Retrieves sections that match now. No seat is reserved.

A state-changing request

```
POST /v1/enrollments
{"studentId":42,"sectionId":"S101"}
```

Still requires an authoritative capacity check.

Takeaway: A filtered result is informational. A later write may conflict because availability changed.

RFC 9110: HTTP semantics

Strict Parsing Beats Silent Coercion

Request value	Why it is a problem	Our response
page=two	Not an integer	400
page=-1	Outside the allowed range	400
page=1.5	Not an integer	400
perPage=101	Above the documented maximum	400
state=opens	Unknown state	400
term=W28	Valid syntax, perhaps no matches	200, items: []

Takeaway: A valid filter with no results is not the same as an invalid filter.

RFC 9110: HTTP semantics | RFC 9457: Problem Details

Arrays, Repeated Keys, and Encoding


```
?term[]=F26              # an array shape, not our scalar contract
?term=F26&term=W27        # ambiguous unless repetition is defined
?sort=+title             # plus often decodes to a space
?sort=-availableSeats%2Cid  # encoded comma, valid after decoding
```

- Define one scalar value per parameter for this teaching API.
- Our raw-query parser rejects duplicates before PHP can collapse them.
- Percent-decode once, validate UTF-8 and types, then build normalized values.

Takeaway: Do not assume every query value is a harmless scalar string.

OpenAPI 3.1.2 | PHP: http_build_query

From Query Text to Typed Values


```php
// A small bounded integer parser (excerpt).
if (!preg_match('/^(0|[1-9][0-9]*)$/D', $raw)) {
    throw new InvalidArgumentException('Use an integer.');
}
if (strlen($raw) > strlen((string) $max)) {
    throw new InvalidArgumentException('Value too large.');
}
$value = (int) $raw;
if ($value < $min || $value > $max) {
    throw new InvalidArgumentException('Outside allowed range.');
}
```

Takeaway: Validate before casting. Otherwise `two` or an extreme value can become an unintended number.

This contract also rejects leading zeros such as `01`. Full reusable parsing code is in the companion examples.

Filter Values Become Bound Parameters


```php
$where = []; $params = [];
if (isset($q['term'])) {
    $where[] = 's.term = :term';
    $params['term'] = $q['term'];
}
if (isset($q['state'])) {
    $where[] = 's.state = :state';
    $params['state'] = $q['state'];
}
if (isset($q['minSeats'])) {
    $where[] = '(s.capacity - s.seats_taken) >= :minSeats';
    $params['minSeats'] = $q['minSeats'];
}
```

Takeaway: Choose SQL structure from trusted code. Pass every client value separately.

PHP: PDO::prepare

Search Is Not Always Exact Filtering

Exact predicate

```
WHERE s.term = :term
```

Useful for term, state, and identifiers.

Text search

```
WHERE c.title LIKE :pattern
```

Requires rules for case, accents, wildcards, ranking, and length.

Takeaway: Prepared statements prevent SQL syntax injection; they do not remove the special meaning of LIKE wildcards.

A q search parameter is not implemented in our small example. Literal %, _, and escape characters need an explicit search policy.

PHP: PDO::prepare | Google AIP-160: filtering

Richer Filters Are a Language Design

Approach	Example	Trade-off
Flat fields	?term=F26&state=open	Easy for a small fixed catalog.
Bracket convention	?filter[state]=open	Namespacing; parser and schema must agree.
Expression language	?filter=state="open"	More powerful; requires grammar and limits.

⚠ Do not execute the filter as SQL

Parse only documented operators, fields, and value types. Bound expression depth, string size, and list length.

Provider conventions differ. Our example deliberately supports only flat fields.

Google AIP-160: filtering | JSON:API: sorting and pagination

Authorization Is Not a Client Filter


```
-- Private enrollment collection: conceptual example only.
WHERE tenant_id = :trustedTenant
  AND student_id = :authenticatedStudent
  AND term = :requestedTerm
ORDER BY id
LIMIT :take OFFSET :skip
```

⚠ The same scope applies everywhere

The item query, total count, facets, next links, and cache entry must not reveal rows outside the caller's permission scope.

Takeaway: Apply trusted access constraints before counting and paging; a hidden UI field is not access control.

Our runnable collection helper uses synthetic public sections and does not implement authentication.

OWASP: REST security

3. Sorting Predictably

Use a complete ordering so page boundaries are explainable

Sort Is an Ordered List of Keys


```
GET /v2/sections?sort=-availableSeats,id
```

Position	Key	Meaning
1	-availableSeats	Higher available-seat count first.
2	id	For equal counts, smaller ID first.

S102: 4 **S103: 4** **S108: 4** **S107: 2** **S101: 1**

Takeaway: The second key resolves ties; it is not another independent sort.

The displayed rows assume the Fall/open/minSeats=1 filters from the running example.

JSON:API: sorting and pagination | MariaDB: ORDER BY

Ties Can Move the Page Boundary

Only availableSeats DESC

```
Valid ordering A: S102 S103 | S108 ...
Valid ordering B: S108 S102 | S103 ...
```

Add a unique tie-breaker

```
ORDER BY availableSeats DESC, id ASC
S102 S103 | S108 S107 | S101
```

Takeaway: A deterministic total order stabilizes ties for unchanged data. It does not freeze a changing database.

MariaDB: ORDER BY | MariaDB: pagination optimization

Build ORDER BY from an Allowlist


```php
$allowed = [
    'id' => 's.id',
    'title' => 'c.title',
    'availableSeats' => '(s.capacity - s.seats_taken)',
];
$descending = str_starts_with($token, '-');
$field = $descending ? substr($token, 1) : $token;
if (!isset($allowed[$field])) {
    throw new InvalidArgumentException('Unknown sort key.');
}
$piece = $allowed[$field] . ($descending ? ' DESC' : ' ASC');
```

Takeaway: You can bind values, not column names or ASC/DESC keywords. Never paste raw sort text into SQL.

Full helper rejects duplicate keys, limits key count, and appends id when necessary.

PHP: PDO::prepare

The Sort Contract Has Hidden Decisions

Decision	Why a client cares
Default ordering	Requests without sort must still be predictable.
Unique tie-breaker	Equal titles or seat counts need a stable order.
Case/accent collation	Algebra, algebra, and accented names may compare differently.
Null placement	Unknown prices or dates need a defined location.
Mutable keys	A changing seat count can move a row between pages.

Takeaway: Our seed titles are non-null ASCII. Production language-sensitive sorting needs a documented collation.

MariaDB: ORDER BY

Sorting Checkpoint: Predict Page 1

Five matching rows

S101 has 1 seat; S102, S103, and S108 have 4; S107 has 2. Sort by `-availableSeats,id`; request two items.

S102 **S103** **S108** **S107** **S101**

Answer

Page 1 contains S102 and S103. S108 is the next row because its ID follows theirs among rows with four seats.

4. Pagination and Navigation

Bound work and payloads without
pretending one page is the whole collection

Why a Collection Needs a Page Contract

For the client

- Readable results on a phone or projector.
- Lower transfer and rendering costs.
- Progressive navigation or infinite scroll.

For the server

- Bounded response size.
- Less memory held per request.
- A foundation for work and rate limits.

Takeaway: A small page limits output, not necessarily the database work required to find it.

Google AIP-158: pagination | MariaDB: pagination optimization

Offset and Page Numbers


```
GET /v2/sections?page=2&perPage=2
```

A one-based page contract

$$\text{offset} = (\text{page} - 1) \times \text{perPage} = (2 - 1) \times 2 = 2.$$

Skip two matching ordered rows; return at most two more.

S102 **S103** **S108** **S107** **S101**

Filtering and sorting have already selected the ordered sequence shown here.

MariaDB: LIMIT

The Three Pages of Our Running Query

Request	Items	Returned / total
page=1, perPage=2	S102, S103	2 / 5
page=2, perPage=2	S108, S107	2 / 5
page=3, perPage=2	S101	1 / 5

```
Same filters each time: term=F26&state=open&minSeats=1
Same ordering each time: sort=-availableSeats,id
```

Takeaway: Page size is a maximum. A final page may contain fewer items.

Return Navigation and Useful Metadata


```
{
  "items": [{"id":"S108", "...":"..."}, {"id":"S107", "...":"..."}],
  "page": {
    "number": 2, "perPage": 2, "returned": 2,
    "totalItems": 5, "totalPages": 3
  },
  "links": {
    "self": "/v2/sections?...&page=2&perPage=2",
    "next": "/v2/sections?...&page=3&perPage=2",
    "prev": "/v2/sections?...&page=1&perPage=2"
  }
}
```

Takeaway: returned describes this response; totalItems describes the complete matching set at counting time.

Illustrative shortened body. The companion schema and complete response contain all fields and fully encoded links.

Boundary Cases Are Contract Decisions

Condition	Our explicit policy
Missing page / perPage	Use page 1 and perPage 20.
No matching sections	200, items: [], totalItems: 0, totalPages: 0.
Page beyond the last page	200, empty items, next: null; prev can lead to the last valid page.
Negative or excessive values	400; do not silently change this contract.
Unknown member /sections/NOPE	404: different from an empty collection.

Takeaway: The current page field stays at least 1 even when totalPages is 0.

At the maximum page budget, next is null and page.traversalLimited is true if more matches exist. A budget stop is not full exhaustion.

RFC 9110: HTTP semantics

Preserve Filters and Sort in Every Link

Broken next link

```
/v2/sections?page=2
```

The server lost term, state, minSeats, sort, and page size.

A link to the same query

```
/v2/sections?term=F26&state=open
&minSeats=1&sort=-availableSeats%2Cid
&page=2&perPage=2
```

Only the page number changes.

Takeaway: Changing a filter or sort in the UI starts a new traversal at page 1.

Google AIP-158: pagination | RFC 8288: Web Linking

Build Links Instead of Concatenating Input


```
function pageLink(array $normalized, int $page): string
{
    $normalized['page'] = $page;
    return '/v2/sections?' . http_build_query(
        $normalized, '', '&', PHP_QUERY_RFC3986
    );
}
```

- Use the validated, normalized parameter set.
- Percent-encode query values. Use a fixed base path or configured public origin.
- Do not trust an arbitrary incoming Host field when building public absolute links.

Takeaway: Do not append a second page= key to an existing raw query.

PHP: http_build_query | OWASP: REST security

Navigation in the HTTP Link Header


```
HTTP/1.1 200 OK
Content-Type: application/json
Link: </v2/sections?term=F26&page=3&perPage=2>; rel="next",
      </v2/sections?term=F26&page=1&perPage=2>; rel="prev"

{"items":[...], "page":{...}, "links":{...}}
```

- The relation says what the target URI means.
- A client should follow the supplied next link rather than guess its syntax.
- A next relation may be absent when there is no next page.

Header continuation is visual only. OpenAPI Link Objects are a separate documentation construct, not these HTTP fields.

RFC 8288: Web Linking | GitHub: pagination

A Page Is Usually Still 200 OK

Application pagination

```
GET /v2/sections?page=2

200 OK
{"items":[...],"links":{...}}
```

HTTP range transfer

```
GET /large-video.mp4
Range: bytes=0-999

206 Partial Content
```

Takeaway: Do not use 206 merely because the JSON contains one page. Range semantics are a different HTTP feature.

RFC 9110: HTTP semantics

Push Filtering, Sorting, and Paging into SQL


```
SELECT s.id, c.title, s.term, s.state,
       (s.capacity - s.seats_taken) AS availableSeats
FROM sections s JOIN courses c ON c.code = s.course_code
WHERE s.term = :term
  AND s.state = :state
  AND (s.capacity - s.seats_taken) >= :minSeats
ORDER BY (s.capacity - s.seats_taken) DESC, s.id ASC
LIMIT :take OFFSET :skip
```

Takeaway: Fetch the requested slice, not all records followed by array_slice in PHP.

This statement has exactly one row per section. Authorization predicates would belong in WHERE as well.

MariaDB: LIMIT | PHP: PDO::prepare

Bind LIMIT and OFFSET as Integers


```php
$stmt = $db->prepare($sql);
$stmt->bindValue(':term', 'F26', PDO::PARAM_STR);
$stmt->bindValue(':state', 'open', PDO::PARAM_STR);
$stmt->bindValue(':minSeats', 1, PDO::PARAM_INT);
$stmt->bindValue(':take', $q['perPage'], PDO::PARAM_INT);
$stmt->bindValue(':skip',
    ($q['page'] - 1) * $q['perPage'], PDO::PARAM_INT);
$stmt->execute();
$rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

Takeaway: Use explicit integer binding for numeric SQL limits. The values were already bounded during validation.

PHP: PDO::prepare | PHP: bindValue

Count the Same Matching Set


```sql
SELECT COUNT(*)
FROM sections s JOIN courses c ON c.code = s.course_code
WHERE s.term = :term AND s.state = :state
  AND (s.capacity - s.seats_taken) >= :minSeats
-- No LIMIT or OFFSET in this count.
```

- Use the same filters and authorization scope as the item query.
- Do not use count(items) as totalItems.
- Do not use PDO rowCount() to count SELECT results.

Takeaway: Two separate reads can observe different states under concurrent updates; our example does not promise a snapshot.

PHP: rowCount

Joins Can Accidentally Multiply Resources

A one-to-many join

```
Section S102 -> enrollment 1
Section S102 -> enrollment 2
Section S103 -> enrollment 3
```

An accidental page of joined rows

```
LIMIT 2 returns:
S102, S102

Only one distinct section!
```

Takeaway: Page unique resource identities, not an expanded join. Use EXISTS, aggregation, or a two-stage fetch.

COUNT(*) over the expanded join also counts enrollment rows, not sections. The supplied course JOIN is many-to-one.

Offset Paging Under Concurrent Changes

Before: first request returns A, B



A new X appears before A; page 2 still skips two rows

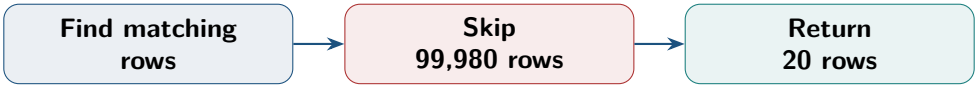


Page 2 now returns B, C: B is repeated.

Takeaway: Insertions shift offsets. Deletions before the boundary can cause a skipped row.

MariaDB: pagination optimization

Deep Offsets: More Work Than Results




```
SELECT ... ORDER BY id LIMIT 20 OFFSET 99980
```

- A database may still scan or step past many matching entries.
- Indexes help, but a small response does not imply a cheap query.
- Measure actual plans and latency; do not promise universal complexity.

MariaDB: pagination optimization | MariaDB: LIMIT

5. Cursors and Keyset Pagination

Continue after a position instead of counting past earlier results

A Cursor Is a Continuation Instruction

Offset question

"Skip the first 40 rows of the result as it exists now."

Keyset question

"Continue after the last ordering key I received."

```
GET /v2/sections-feed?term=F26&limit=2&after=opaque-token
```

Takeaway: Cursor is an API contract; keyset is one database strategy behind it. A cursor is not necessarily a database cursor.

sections-feed is a separate design example. The companion executable helpers implement page/perPage only.

Start with One Stable Unique Sort Key




```sql
-- Last returned ID was S102; ordering is id ASC.
SELECT ... FROM sections
WHERE term = :term AND id > :lastId
ORDER BY id ASC
LIMIT :takePlusOne
```

Why this works

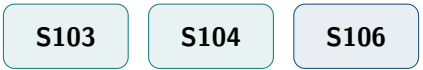
A new row before S102 does not change the boundary. We continue after an identity, not after a count.

MariaDB: pagination optimization

Look Ahead Without Losing a Row


```
Requested limit = 2
Database returns up to 3: [S103, S104, S106]

Return to client: [S103, S104]
hasNext: true
Next boundary: S104 (the last RETURNED row)
```



S106 is lookahead only; it must remain eligible for the next request.

Takeaway: With a strict greater-than predicate, using the lookahead row as the boundary would skip it.

There are other valid inclusive-boundary designs. Do not mix their token rules with this strict-after design.

MariaDB: pagination optimization

Composite Sorts Need Composite Cursors


```sql
-- Sort: availableSeats DESC, id ASC
-- Last returned tuple: (4, 'S103')
WHERE (availableSeats < :lastSeats
   OR (availableSeats = :lastSeatsAgain AND id > :lastId))
ORDER BY availableSeats DESC, id ASC
LIMIT :takePlusOne
```

Apply the whole tuple

The next matching row can be S108 with the same 4 seats, then rows with fewer than 4. Using only the seat count would skip tied rows.

Takeaway: Use different named placeholders for repeated values with native PDO prepares.

Use the actual expression or a derived table, not a SELECT alias in WHERE. When adding filters, keep this entire OR predicate in parentheses.

PHP: PDO::prepare | MariaDB: ORDER BY

Tokens Must Not Become Permissions

Token concern	Required design decision
Meaning	Boundary plus order/filter/version context.
Tampering	Validate format and bounds; use a server lookup or signature when needed.
Confidentiality	Base64 is encoding, not encryption. A signature does not hide data.
Changed query	Reject incompatible filters/sort or restart traversal.
Authorization	Re-check permissions on every request.
Expired/invalid token	Document an error and restart policy.

Takeaway: A continuation token is not a credential and does not authorize access to any row.

Google AIP-158: pagination | OWASP: REST security

Keyset Pagination Is Not a Snapshot

What it can improve

- Avoids shifts from earlier insertions and deletions.
- Seeks efficiently with a suitable index.
- Fits feeds with stable ordering keys.

What still changes

- Sort-key updates cross boundaries.
- New rows after the boundary may appear.
- Deleted rows disappear.
- Access and filter membership can change.

Takeaway: A repeatable export needs a snapshot, a materialized result, or suitable versioned data.

A created-before timestamp alone does not freeze later updates or deletions.

MariaDB: pagination optimization

Choose the Strategy for the User Task

User task	Useful starting point	Important caveat
Small school catalog	Page/offset	Simple numbered navigation; data can shift.
Rapidly growing activity feed	Keyset + cursor	Stable key and context-bound token.
Jump to an arbitrary page	Page/offset or extra index	Keyset has no natural page number.
Financial/reporting export	Snapshot/job design	Pagination alone cannot ensure a fixed dataset.
Backward navigation	Explicit before/prev design	Reverse comparison and order carefully.

Takeaway: Neither offset nor cursor is universally best. State the guarantees, not just the parameter name.

Cursor Checkpoint

The current response

```
Order: id ASC
Returned: [S103, S104]
Lookahead exists: S106
Token boundary: S104
```

Predict the next request

Use a strict condition `id > S104`. S106 remains eligible. A new S100 will not be reinserted into this forward traversal.

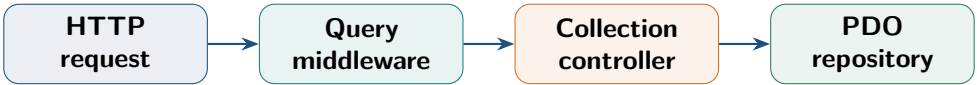
A harder question

If the order is availableSeats and S104 changes its seat count, no fixed-snapshot guarantee follows from the cursor.

6. Contract to Implementation

Connect middleware, OpenAPI, validated queries, SQL, and response links

Our End-to-End Responsibility Map



Layer	One main responsibility
Middleware	Reject invalid raw query; attach normalized query attributes.
Controller	Call repository; create response and headers.
Repository	Apply trusted predicates, total order, LIMIT/OFFSET, and mapping.
Emitter	Send the returned status, headers, and body once.

Takeaway: Generic normalization is reusable. Section-specific SQL stays with the repository.

Document Query Parameters in OpenAPI


```yaml
openapi: 3.1.2
info: {title: Section Collection API, version: 2.0.0}
servers:
  - url: http://127.0.0.1:8000/v2
paths:
  /sections:
    get:
      parameters:
        - {name: page, in: query,
           schema: {type: integer, minimum: 1, maximum: 10000, default: 1}}
        - {name: perPage, in: query,
           schema: {type: integer, minimum: 1, maximum: 100, default: 20}}
      # responses and the other parameters follow
```

Takeaway: The parameter schema describes the logical value. The wire query still arrives as text.

OpenAPI 3.1.2

Describe Sort Syntax and Runtime Rules


```
- name: sort
  in: query
  description: >-
    Comma-separated keys: id, title, availableSeats.
    Prefix a key with - for descending. Reject repeated fields.
    Append id ascending if id is absent. Maximum three keys.
  schema:
    type: string
    default: id
    maxLength: 64
  example: -availableSeats,id
```

- Descriptions explain rules that a basic string schema does not fully express.
- The implementation must enforce duplicates, allowed fields, and ordering limits.

OpenAPI 3.1.2

Describe the Collection Response


```
SectionPage:
  type: object
  required: [items, page, links]
  properties:
    items: {type: array, items: {$ref: '#/components/schemas/Section'}}
    page: {$ref: '#/components/schemas/PageMetadata'}
    links:
      type: object
      required: [self, next, prev]
      properties:
        self: {type: string, format: uri-reference}
        next: {type: [string, 'null'], format: uri-reference}
        prev: {type: [string, 'null'], format: uri-reference}
```

Takeaway: In OpenAPI 3.1, nullable next/prev use a union type. Documentation alone does not generate these links.

OpenAPI 3.1.2

A Useful Invalid-Query Response


```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "about:blank",
  "title": "Bad Request",
  "status": 400,
  "detail": "perPage must be an integer from 1 through 100.",
  "invalidParams": [{"name":"perPage", "reason":"above maximum"}]
}
```

Takeaway: invalidParams is our documented extension, not a required RFC field. Do not echo unsafe secrets or raw SQL.

RFC 9457: Problem Details

Attach Normalized Values to the Request


```php
// Collection-route middleware, inside safe error conversion.
try {
    $query = CollectionQuery::parse($request->getUri()->getQuery());
} catch (InvalidArgumentException $error) {
    return problemResponse(400, $error->getMessage());
}
return $next->handle($request->withAttribute('collectionQuery', $query));
```

```text
// Normalized values:
{"term":"F26","state":"open","minSeats":1,
 "sort":"-availableSeats,id","page":2,"perPage":2}
```

Takeaway: Downstream code consumes validated values; it does not repeatedly parse the raw query.

Illustrative PSR-15 method body. The reusable CollectionQuery helper is complete in the companion.

PHP-FIG: PSR-7 | PHP-FIG: PSR-15

The Controller Stays Small


```php
$query = $request->getAttribute('collectionQuery');
$payload = $repository->page($query);
$json = json_encode($payload,
    JSON_THROW_ON_ERROR | JSON_UNESCAPED_SLASHES);
$response->getBody()->write($json);
return $response->withStatus(200)
    ->withHeader('Content-Type', 'application/json')
    ->withHeader('Cache-Control', 'no-store');
```

What is deliberately absent

No raw SQL, no user-selected column names, no echo/exit, and no database credentials in the response.

Handler excerpt; no-store is a conservative teaching policy, not a claim that every collection should be uncached.

PHP-FIG: PSR-7 | RFC 9111: caching

Put Together the Exact Running Query


```
GET /v2/sections?term=F26&state=open&minSeats=1
    &sort=-availableSeats,id&page=2&perPage=2

Parsed: page=2, perPage=2, offset=2
Predicate: F26 AND open AND availableSeats >= 1
Matching order: S102, S103, S108, S107, S101
Page: S108, S107
Metadata: returned=2, totalItems=5, totalPages=3
Next: same query, page=3
```

Takeaway: Every number in this trace is derived from the shared dataset, not from a decorative mockup.

Filtering and Paging Affect Cache Identity

These are different requests

```
/v2/sections?term=F26&page=1
/v2/sections?term=W27&page=1
/v2/sections?term=F26&page=2
```

A correct application cache key

- Normalized filters.
- Sort and pagination boundary.
- Representation variants.
- Trusted authorization scope, when applicable.

Takeaway: Vary handles nominated request headers; it is not the mechanism for distinguishing query parameters.

A shared cache must not leak personalized results. The supplied example uses no-store and makes no caching claim.

RFC 9111: caching

Indexes Follow Actual Query Shapes


```sql
CREATE INDEX term_state_id ON sections(term, state, id);

-- A query that can benefit from that ordering:
WHERE term = :term AND state = :state
  AND id > :lastId
ORDER BY id ASC
LIMIT :takePlusOne
```

- Equality filters, then ordered boundary fields, can fit a useful index.
- Sorting by title from a joined table is a different query shape.
- Sorting by capacity minus seats_taken may still require a computed sort.

Takeaway: One index does not accelerate every optional filter and sort combination. Inspect EXPLAIN and measure.

MariaDB: pagination optimization | MariaDB: ORDER BY

Bound Work, Not Only the Page Size

Control	Problem it helps contain
Maximum page size	Huge payloads and PHP result arrays.
Maximum offset / cursor traversal	Expensive deep browsing and scraping.
Approved filters and sort keys	Arbitrary expensive or sensitive queries.
String/list/depth limits	Unbounded search complexity.
Database timeouts / rate limits	Repeated expensive work.
Optional or estimated totals	COUNT over a very large matching set.

Takeaway: A count query can cost more than fetching the next page. Estimates must be labeled.

Google AIP-158: pagination | OWASP: REST security

Do Not Fetch the Same Parent in a Loop

N+1 query pattern

```
1 query: get 20 section rows
20 queries: fetch each course title
Total: 21 queries
```

A suitable JOIN or batch

```
1 query: sections JOIN courses
Total: 1 query for this view
```

Takeaway: Avoid unnecessary round trips, but do not replace N+1 with a one-to-many JOIN that duplicates the paged resources.

A Client Follows Pages, Not Guesses


```php
$url = $firstUrl; $seen = [];
for ($step = 0; $url !== null && $step < 100; $step++) {
    if (isset($seen[$url])) throw new RuntimeException('Link loop');
    $seen[$url] = true;
    $page = fetchJsonFromTrustedApi($url);
    foreach ($page['items'] as $item) { consume($item); }
    $url = $page['links']['next'];
    if ($page['page']['traversalLimited'] ?? false)
        throw new RuntimeException('Server traversal limit');
}
if ($url !== null) throw new RuntimeException('Client page budget');
```

Takeaway: Follow the declared end condition, not a guessed total or the assumption that every page is full.

Client algorithm, not a complete HTTP library. The abstract fetch helper resolves relative links and rejects an untrusted origin before sending credentials.

Google AIP-158: pagination | GitHub: pagination

7. Practice and Verification

Use examples, failure cases, and observable evidence

A Real Catalog to Explore


```
https://api.artic.edu/api/v1/artworks
  ?page=1&limit=2&fields=id,title,api_link
```

Change / inspect	What it illustrates
page=1 then page=2	A new slice of a collection.
limit=2	A bounded result size.
fields=id,title,api_link	Field selection, not row filtering.
pagination.next_url	Server-supplied navigation.

Takeaway: The Art Institute uses its own parameter vocabulary. Read the contract rather than assume our perPage syntax.

Public-service documentation was checked; no current live response is asserted. Rehearse before class or use the synthetic dataset.

Art Institute: API docs

GitHub: Navigation Can Live in Headers


```
GET /repos/octocat/Spoon-Knife/issues?per_page=2&page=1
Host: api.github.com
Accept: application/vnd.github+json

Link: <...page=2>; rel="next", <...>; rel="last"
```

- Inspect Link, not just the JSON body.
- Follow the URL whose relation is next.
- Rate limits, data, and available relations can change.
- Issue-list endpoints can also include pull requests.

Illustrative response structure; no live issue count or exact last-page value is promised.

GitHub: pagination | GitHub: issues endpoints

Same Concepts, Different Conventions

Source	Pagination	Sorting / filtering
Our teaching API	page + perPage	Approved sort keys; term, state, minSeats filters.
GitHub REST	page + per_page for many lists	Endpoint-specific filters and sorting.
Google AIPs	page_size + page_token	order_by and an optional filter language.
JSON:API	page parameter family	sort with comma keys and - direction.

Takeaway: Do not mix conventions without a contract. The shared ideas matter more than one spelling.

Google AIP-132: list and ordering | Google AIP-158: pagination | JSON:API: sorting and pagination

Tests for the Query Contract

Test	Expected result
No parameters	page 1, perPage 20, sort id.
F26 + open + minSeats=1	Five matching sections.
Main query, page 2, perPage 2	S108 then S107.
sort=title,title	400: repeated sort field.
page=0 / perPage=101	400: documented bounds.
term[]=F26 / repeated term	400: unsupported shape or duplicate key.
Valid empty term selection	200 and an empty collection.

Takeaway: Use a fixed fixture so expected rows, order, and totals are checkable.

Tests for Behavior Across Pages

Property	What to verify
Completeness on a fixed fixture	All matching IDs appear exactly once across pages.
Ordering	Every adjacent pair follows all declared sort keys.
Link preservation	Next keeps filters, sort, and page size.
Boundary handling	Empty, final, and beyond-last pages behave as documented.
Query changes	Changing filters restarts pagination.
Concurrent data	Insert/delete/mutable-key cases do not imply a snapshot.

Takeaway: A successful first page is not enough to verify a collection API.

Tests for the Middleware Stack

Case	Expected evidence
Successful handler	Before/after trace has reverse response order.
Early validator rejection	Handler and repository are not called.
Handler exception	Safe error response; cleanup still runs.
Route-specific policy	Unrelated routes remain unaffected.
Repeated requests	No request attributes leak into a later request.
Final response	Required headers are present even on intended error paths.

Takeaway: Tests must exercise both returned errors and thrown exceptions.

PHP-FIG: PSR-15

Diagnose the Failure at the Right Layer

Symptom	Inspect first
Only two rows; wrong total	Count predicate and pagination envelope.
Same row repeats with unchanged data	Missing unique sort tie-breaker.
Rows shift after an insertion	Offset behavior; not necessarily a parser bug.
SQL error near LIMIT	Integer binding and driver configuration.
Error lacks request ID	Exception bypassed inner after-code.
Next page loses filter	Link builder used an incomplete parameter set.
Private counts exposed	Authorization scope missing from count query.

Takeaway: Trace the request through middleware, normalization, query plan, and output instead of changing random code.

Design Exercise: An Online Store

A shopper's goal

Find available headphones under a chosen price, order cheapest first, and browse ten at a time. Two products can have the same price.

1. Name the resource and choose a documented filter vocabulary.
2. Define the numeric price unit: dollars, cents, or a decimal string.
3. Choose a total order; include a unique product ID tie-breaker.
4. Specify page-size limits, invalid filters, and next links.
5. Explain what happens if a price changes between requests.

Takeaway: Changing the domain should not change the underlying design method.

One Possible Store Contract

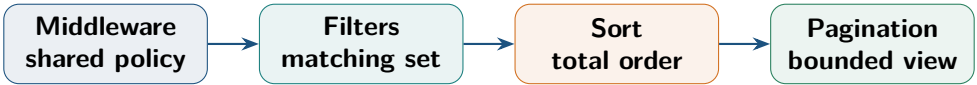

```
GET /products?category=headphones&inStock=true
    &maxPriceCents=15000&sort=priceCents,id&page=1&perPage=10

Meaning: stocked headphones priced at most 150 CAD,
ordered by price, then product ID. Prices are integer cents.
Response: items + metadata + links; invalid query -> 400.
```

A deliberate limitation

This is a design example, not an implementation or a claim about a real store. Price changes can still move products between pages.

The Week 5 Connections to Remember



- Validate values; allowlist SQL structure.
- Apply access scope and filters before ordering and paging.
- Keep count, items, and links aligned with the same query.
- A cursor reduces some drift; it does not freeze the world.
- The contract, PHP behavior, and observed data must agree.

From This Week to the Next

Now

- Shared request processing.
- Precise collection queries.
- Deterministic ordering.
- Page/offset and cursor trade-offs.

Next layers

- Content negotiation and representations.
- Consistent operational error handling.
- Structured logging.
- Richer security and quality assurance.

Takeaway: Keep concerns separate, and extend the specification whenever observable behavior changes.

8. Reference and Further Reading

Complete query examples, compact lookups, and primary sources

Implementation Reference: Query Builder


```php
$plan = CollectionQuery::plan($normalized);
// countSql and sql share exactly the same FROM + WHERE.
$count = $db->prepare($plan['countSql']);
foreach ($plan['filters'] as $key => $value) {
    $count->bindValue(':' . $key, $value,
        is_int($value) ? PDO::PARAM_INT : PDO::PARAM_STR);
}
$count->execute();
$total = (int) $count->fetchColumn();
// Execute plan.sql with plan.params using the same binding pattern.
// Map public fields; do not expose raw storage rows.
return CollectionQuery::envelope($items, $normalized, $total);
```

Takeaway: The companion contains complete CollectionQuery and SectionRepository classes, not only these excerpts.

Two ordinary reads do not guarantee a shared snapshot. MariaDB integration execution is reported separately from pure/query tests.

Vocabulary: The Minimum Shared Language

Term	Meaning
Middleware	Reusable component surrounding request handling.
Short circuit	Return before delegating to inner processing.
Predicate	A condition deciding which rows match.
Total order	Every matching row has a deterministic relative position.
Offset	Number of ordered matching rows to skip.
Keyset	Continue after a previously returned ordering tuple.
Cursor token	A protocol value identifying continuation context.
Snapshot	A stable view of data, stronger than ordinary pagination.

References: Middleware and PHP

- PHP-FIG: PSR-7
- PHP-FIG: PSR-15
- Slim 4: middleware order
- Slim 4: error middleware
- Slim 4: installation
- PHP: PDO::prepare
- PHP: bindValue
- PHP: http_build_query

PSR-15 classes require the appropriate interfaces and message implementation. Framework registration order is not universal.

References: Collection Design

- Google AIP-158: pagination
- Google AIP-160: filtering
- Google AIP-132: list and ordering
- JSON:API: sorting and pagination
- MariaDB: pagination optimization
- MariaDB: LIMIT
- MariaDB: ORDER BY
- GitHub: pagination

Google AIPs and JSON:API define their own conventions. Our teaching API is not advertised as conforming to either profile.

References: Contracts, HTTP, and Assets

- OpenAPI 3.1.2
- RFC 9110: HTTP semantics
- RFC 8288: Web Linking
- RFC 9111: caching
- RFC 9457: Problem Details
- OWASP: REST security
- Art Institute: API docs

All diagrams and UI illustrations are original TikZ vectors. Icons use Font Awesome Free via fontawesome5; license. No separate image downloads are needed. Primary-source references checked 25 September 2026.